# Rapport sur les microservices de commande

Atelier Java JEE et Spring Boot avec RabbitMQ et Kafka

## 1 Rôle des microservices

Order Service valide la demande, attribue un identifiant et publie l'événement OrderCreated. Notification Service consomme l'événement RabbitMQ et simule l'envoi d'une notification. La séparation permet de déployer les services indépendamment et d'accepter des commandes pendant un arrêt du consommateur.

## 2 REST synchrone et événement asynchrone

Dans un appel REST synchrone, le client attend la réponse HTTP. Dans le flux asynchrone, Order Service attend la confirmation de publication de RabbitMQ, puis répond au client. Notification Service traite le message plus tard et l'acquitte séparément.

## Test de création et de notification

L'appel POST /api/orders avec un montant de 149,90 a renvoyé HTTP 201. Le même orderId apparaît dans la réponse et dans le log de notification. À l'arrêt du consommateur, la commande est restée en file puis a été traitée après son redémarrage.

**Figure 1. Appel REST réussi**

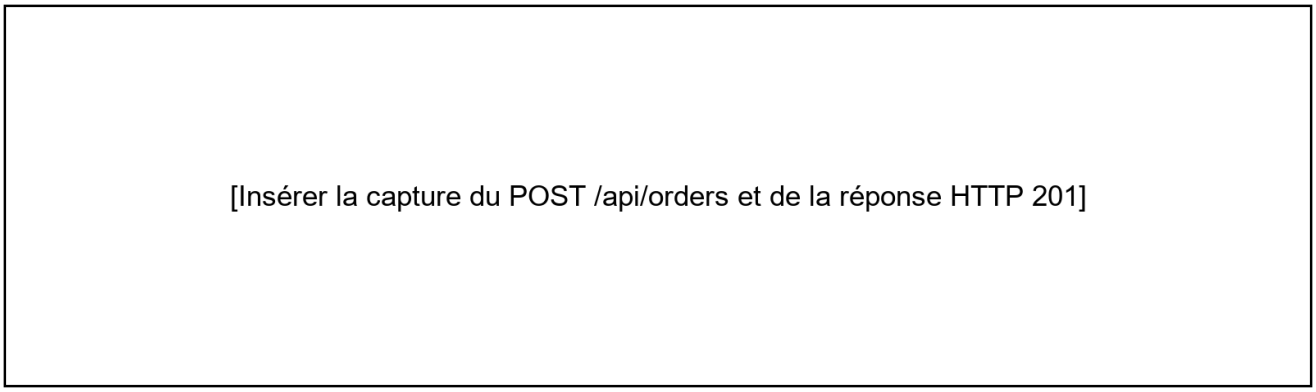
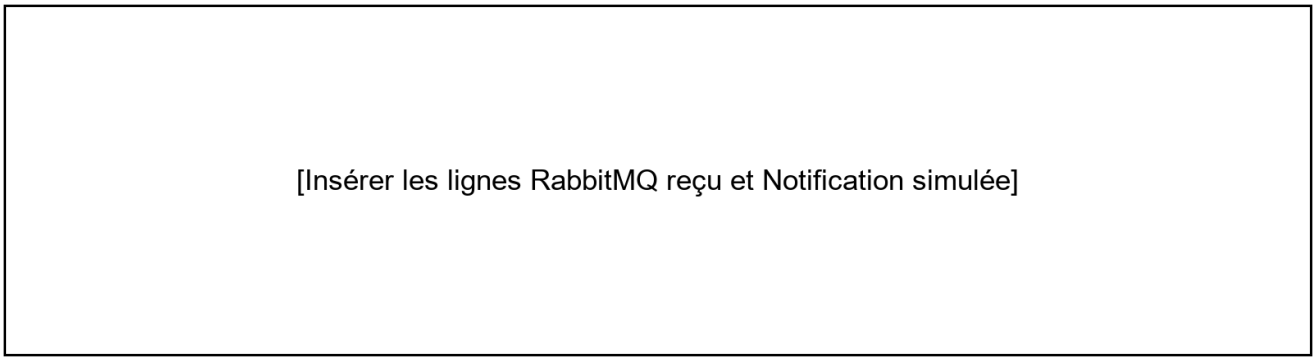



**Figure 2. Logs de notification**



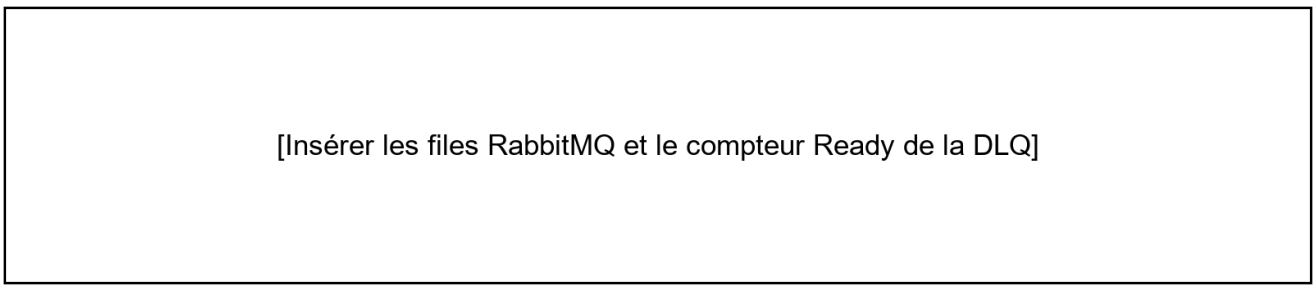
## 3 Routage et acquittement RabbitMQ

L’exchange direct orders.exchange dirige les messages selon la routing key order.created. Le binding les achemine vers la file durable orders.notifications, qui les conserve jusqu’au traitement. L’ACK confirme la réussite du traitement et permet au broker de retirer le message.

## 4 Échec de traitement et reprise

Une commande de montant 13 a été acceptée avec HTTP 201, puis le consommateur a échoué trois fois à 300 ms d’intervalle. Le dernier rejet a envoyé le message dans orders.notifications.dlq, dont le compteur Ready a augmenté de un. Après correction, le message peut être rejoué. Un arrêt brutal avant l’ACK peut déclencher une redélivrance.

**Figure 3. File RabbitMQ et test d’erreur**



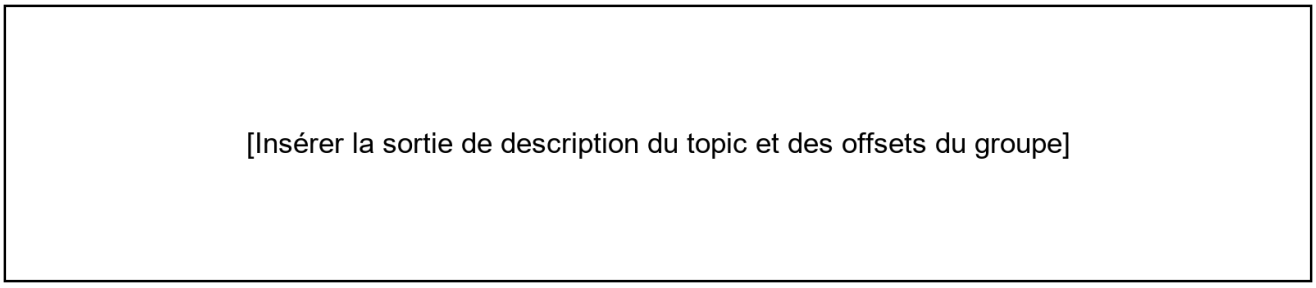

## 5 Topic partition offset et groupe Kafka

Un topic est un journal d’événements divisé en partitions. Un offset indique la position d’un événement dans sa partition. Les consommateurs d’un même groupe se partagent les partitions et enregistrent leurs positions de reprise. Le topic commandes-creees compte trois partitions.

## 6 Clé de partition et ordre

Utiliser orderId comme clé regroupe les événements d’une commande dans la même partition. La clé répétée a été observée sur une seule partition avec des offsets distincts. L’ordre s’applique au sein de chaque partition.

**Figure 4. Partitions et offsets Kafka**




## 7 Deux mesures contre les doublons

Enregistrer chaque orderId dans un registre persistant avec une contrainte d’unicité. Pour un e-mail réel, transmettre le même identifiant comme clé d’idempotence à un fournisseur qui la prend en charge.

## 8 Conclusion et choix du broker

RabbitMQ convient aux tâches à distribuer avec ACK, reprises et file d’échec. Kafka convient à un journal rejouable lu par plusieurs groupes. Dans cet atelier, RabbitMQ traite la notification et Kafka conserve les événements pour la relecture.